

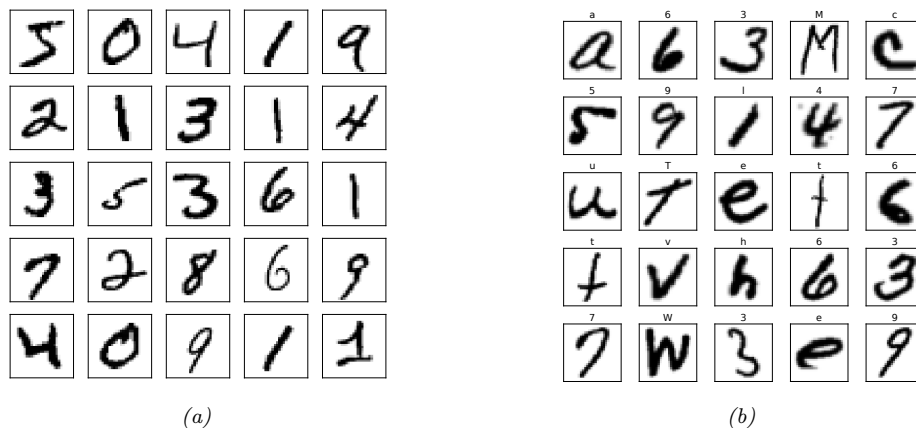


Figure 1.12: (a) Visualization of the MNIST dataset. Each image is 28×28 . There are 60k training examples and 10k test examples. We show the first 25 images from the training set. Generated by [mnist_viz_tf.ipynb](#). (b) Visualization of the EMNIST dataset. There are 697,932 training examples, and 116,323 test examples, each of size 28×28 . There are 62 classes (a-z, A-Z, 0-9). We show the first 25 images from the training set. Generated by [emnist_viz_jax.ipynb](#).

1.5 Data

Machine learning is concerned with fitting models to data using various algorithms. Although we focus on the modeling and algorithm aspects, it is important to mention that the nature and quality of the training data also plays a vital role in the success of any learned model.

In this section, we briefly describe some common image and text datasets that we will use in this book. We also briefly discuss the topic of data preprocessing.

1.5.1 Some common image datasets

In this section, we briefly discuss some image datasets that we will use in this book.

1.5.1.1 Small image datasets

One of the simplest and most widely used is known as **MNIST** [LeC+98; YB19].⁹ This is a dataset of 60k training images and 10k test images, each of size 28×28 (grayscale), illustrating handwritten digits from 10 categories. Each pixel is an integer in the range $\{0, 1, \dots, 255\}$; these are usually rescaled to $[0, 1]$, to represent pixel intensity. We can optionally convert this to a binary image by thresholding. See Figure 1.12a for an illustration.

MNIST is so widely used in the ML community that Geoff Hinton, a famous ML researcher, has called it the “drosophila of machine learning”, since if we cannot make a method work well on MNIST, it will likely not work well on harder datasets. However, nowadays MNIST classification is considered

9. The term “MNIST” stands for “Modified National Institute of Standards”; The term “modified” is used because the images have been preprocessed to ensure the digits are mostly in the center of the image.

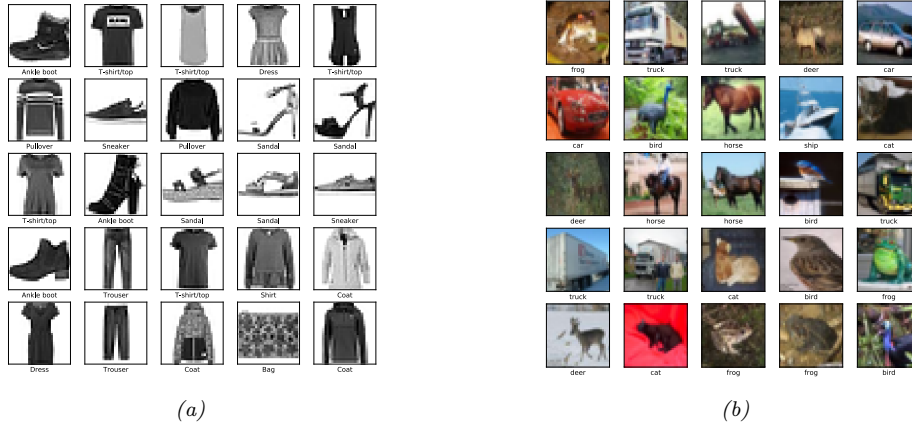


Figure 1.13: (a) Visualization of the Fashion-MNIST dataset [XRV17]. The dataset has the same size as MNIST, but is harder to classify. There are 10 classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle-boot. We show the first 25 images from the training set. Generated by `fashion_viz_tf.ipynb`. (b) Some images from the CIFAR-10 dataset [KH09]. Each image is $32 \times 32 \times 3$, where the final dimension of size 3 refers to RGB. There are 50k training examples and 10k test examples. There are 10 classes: plane, car, bird, cat, deer, dog, frog, horse, ship, and truck. We show the first 25 images from the training set. Generated by `cifar_viz_tf.ipynb`.

“too easy”, since it is possible to distinguish most pairs of digits by looking at just a single pixel. Various extensions have been proposed.

In [Coh+17], they proposed **EMNIST** (extended MNIST), that also includes lower and upper case letters. See Figure 1.12b for a visualization. This dataset is much harder than MNIST, since there are 62 classes, several of which are quite ambiguous (e.g., the digit 1 vs the lower case letter l).

In [XRV17], they proposed **Fashion-MNIST**, which has exactly the same size and shape as MNIST, but where each image is the picture of a piece of clothing instead of a handwritten digit. See Figure 1.13a for a visualization.

For small color images, the most common dataset is **CIFAR** [KH09].¹⁰ This is a dataset of 60k images, each of size $32 \times 32 \times 3$, representing everyday objects from 10 or 100 classes; see Figure 1.13b for an illustration.¹¹

1.5.1.2 ImageNet

Small datasets are useful for prototyping ideas, but it is also important to test methods on larger datasets, both in terms of image size and number of labeled examples. The most widely used dataset

10. CIFAR stands for “Canadian Institute For Advanced Research”. This is the agency that funded labeling of the dataset, which was derived from the TinyImages dataset at <http://groups.csail.mit.edu/vision/TinyImages/> created by Antonio Torralba. See [KH09] for details.

11. Despite its popularity, the CIFAR dataset has some issues. For example, the base error on CIFAR-100 is 5.85% due to mislabeling [NAM21]. This makes any results with accuracy above 94.15% acc suspicious. Also, 10% of CIFAR-100 training set images are duplicated in the test set [BD20].

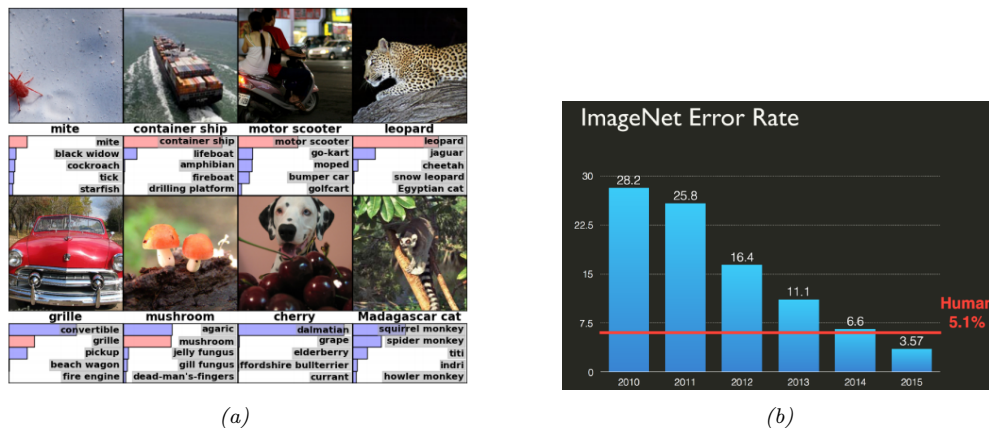


Figure 1.14: (a) Sample images from the **ImageNet** dataset [Rus+15]. This subset consists of 1.3M color training images, each of which is 256×256 pixels in size. There are 1000 possible labels, one per image, and the task is to minimize the top-5 error rate, i.e., to ensure the correct label is within the 5 most probable predictions. Below each image we show the true label, and a distribution over the top 5 predicted labels. If the true label is in the top 5, its probability bar is colored red. Predictions are generated by a convolutional neural network (CNN) called “AlexNet” (Section 14.3.2). From Figure 4 of [KSH12]. Used with kind permission of Alex Krizhevsky. (b) Misclassification rate (top 5) on the ImageNet competition over time. Used with kind permission of Andrej Karpathy.

of this type is called **ImageNet** [Rus+15]. This is a dataset of ~ 14 M images of size $256 \times 256 \times 3$ illustrating various objects from 20,000 classes; see Figure 1.14a for some examples.

The ImageNet dataset was used as the basis of the ImageNet Large Scale Visual Recognition Challenge (**ILSVRC**), which ran from 2010 to 2018. This used a subset of 1.3M images from 1000 classes. During the course of the competition, significant progress was made by the community, as shown in Figure 1.14b. In particular, 2015 marked the first year in which CNNs could outperform humans (or at least one human, namely Andrej Karpathy) at the task of classifying images from ImageNet. Note that this does not mean that CNNs are better at vision than humans (see e.g., [YL21] for some common failure modes). Instead, it mostly likely reflects the fact that the dataset makes many **fine-grained classification** distinctions — such as between a “tiger” and a “tiger cat” — that humans find difficult to understand; by contrast, sufficiently flexible CNNs can learn arbitrary patterns, including random labels [Zha+17a].

Although ImageNet is much harder than MNIST and CIFAR as a classification benchmark, it too is almost “saturated” [Bey+20]. Nevertheless, relative performance of methods on ImageNet is often a surprisingly good predictor of performance on other, unrelated image classification tasks (see e.g., [Rec+19]), so it remains very widely used.

1.5.2 Some common text datasets

Machine learning is often applied to text to solve a variety of tasks. This is known as **natural language processing** or **NLP** (see e.g., [JM20] for details). Below we briefly mention a few text

-
1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
 2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...
-

Table 1.3: We show snippets of the first two sentences from the IMDB movie review dataset. The first example is labeled positive and the second negative. (<UNK> refers to an unknown token.)

datasets that we will use in this book.

1.5.2.1 Text classification

A simple NLP task is text classification, which can be used for **email spam classification**, **sentiment analysis** (e.g., is a movie or product review positive or negative), etc. A common dataset for evaluating such methods is the **IMDB movie review dataset** from [Maa+11]. (IMDB stands for “Internet Movie Database”.) This contains 25k labeled examples for training, and 25k for testing. Each example has a binary label, representing a positive or negative rating. See Table 1.3 for some example sentences.

1.5.2.2 Machine translation

A more difficult NLP task is to learn to map a sentence $\mathbf{x}$ in one language to a “semantically equivalent” sentence $\mathbf{y}$ in another language; this is called **machine translation**. Training such models requires aligned $(\mathbf{x}, \mathbf{y})$ pairs. Fortunately, several such datasets exist, e.g., from the Canadian parliament (English-French pairs), and the European Union (Europarl). A subset of the latter, known as the **WMT dataset** (Workshop on Machine Translation), consists of English-German pairs, and is widely used as a benchmark dataset.

1.5.2.3 Other seq2seq tasks

A generalization of machine translation is to learn a mapping from one sequence $\mathbf{x}$ to any other sequence $\mathbf{y}$. This is called a **seq2seq model**, and can be viewed as a form of high-dimensional classification (see Section 15.2.3 for details). This framing of the problem is very general, and includes many tasks, such as **document summarization**, **question answering**, etc. For example, Table 1.4 shows how to formulate question answering as a seq2seq problem: the input is the text T and question Q , and the output is the answer A , which is a set of words, possibly extracted from the input.

1.5.2.4 Language modeling

The rather grandiose term “**language modeling**” refers to the task of creating unconditional generative models of text sequences, $p(x_1, \dots, x_T)$. This only requires input sentences $\mathbf{x}$, without any corresponding “labels” $\mathbf{y}$. We can therefore think of this as a form of unsupervised learning, which we discuss in Section 1.3. If the language model generates output in response to an input, as in seq2seq, we can regard it as a conditional generative model.

T: In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called “showers”.

Q1: What causes precipitation to fall? A1: **gravity**

Q2: What is another main form of precipitation besides drizzle, rain, snow, sleet and hail? A2: **graupel**

Q3: Where do water droplets collide with ice crystals to form precipitation? A3: **within a cloud**

Table 1.4: Question-answer pairs for a sample passage in the SQuAD dataset. Each of the answers is a segment of text from the passage. This can be solved using sentence pair tagging. The input is the paragraph text T and the question Q . The output is a tagging of the relevant words in T that answer the question in Q . From Figure 1 of [Raj+16]. Used with kind permission of Percy Liang.

1.5.3 Preprocessing discrete input data

Many ML models assume that the data consists of real-valued feature vectors, $\mathbf{x} \in \mathbb{R}^D$. However, sometimes the input may have discrete input features, such as categorical variables like race and gender, or words from some vocabulary. In the sections below, we discuss some ways to preprocess such data to convert it to vector form. This is a common operation that is used for many different kinds of models.

1.5.3.1 One-hot encoding

When we have categorical features, we need to convert them to a numerical scale, so that computing weighted combinations of the inputs makes sense. The standard way to preprocess such categorical variables is to use a **one-hot encoding**, also called a **dummy encoding**. If a variable x has K values, we will denote its dummy encoding as follows: $\text{one-hot}(x) = [\mathbb{I}(x = 1), \dots, \mathbb{I}(x = K)]$. For example, if there are 3 colors (say red, green and blue), the corresponding one-hot vectors will be $\text{one-hot}(\text{red}) = [1, 0, 0]$, $\text{one-hot}(\text{green}) = [0, 1, 0]$, and $\text{one-hot}(\text{blue}) = [0, 0, 1]$.

1.5.3.2 Feature crosses

A linear model using a dummy encoding for each categorical variable can capture the **main effects** of each variable, but cannot capture **interaction effects** between them. For example, suppose we want to predict the fuel efficiency of a vehicle given two categorical input variables: the type (say SUV, Truck, or Family car), and the country of origin (say USA or Japan). If we concatenate the one-hot encodings for the ternary and binary features, we get the following input encoding:

$$\phi(\mathbf{x}) = [1, \mathbb{I}(x_1 = S), \mathbb{I}(x_1 = T), \mathbb{I}(x_1 = F), \mathbb{I}(x_2 = U), \mathbb{I}(x_2 = J)] \quad (1.34)$$

where x_1 is the type and x_2 is the country of origin.

This model cannot capture dependencies between the features. For example, we expect trucks to be less fuel efficient, but perhaps trucks from the USA are even less efficient than trucks from Japan. This cannot be captured using the linear model in Equation (1.34) since the contribution from the country of origin is independent of the car type.

We can fix this by computing explicit **feature crosses**. For example, we can define a new composite feature with 3×2 possible values, to capture the interaction of type and country of origin. The new model becomes

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) \quad (1.35)$$

$$\begin{aligned} &= w_0 + w_1 \mathbb{I}(x_1 = S) + w_2 \mathbb{I}(x_1 = T) + w_3 \mathbb{I}(x_1 = F) \\ &+ w_4 \mathbb{I}(x_2 = U) + w_5 \mathbb{I}(x_2 = J) \\ &+ w_6 \mathbb{I}(x_1 = S, x_2 = U) + w_7 \mathbb{I}(x_1 = T, x_2 = U) + w_8 \mathbb{I}(x_1 = F, x_2 = U) \\ &+ w_9 \mathbb{I}(x_1 = S, x_2 = J) + w_{10} \mathbb{I}(x_1 = T, x_2 = J) + w_{11} \mathbb{I}(x_1 = F, x_2 = J) \end{aligned} \quad (1.36)$$

We can see that the use of feature crosses converts the original dataset into a **wide format**, with many more columns.

1.5.4 Preprocessing text data

In Section 1.5.2, we briefly discussed text classification and other NLP tasks. To feed text data into a classifier, we need to tackle various issues. First, documents have a variable length, and are thus not fixed-length feature vectors, as assumed by many kinds of models. Second, words are categorical variables with many possible values (equal to the size of the vocabulary), so the corresponding one-hot encodings will be very high-dimensional, with no natural notion of similarity. Third, we may encounter words at test time that have not been seen during training (so-called **out-of-vocabulary** or **OOV** words). We discuss some solutions to these problems below. More details can be found in e.g., [BKL10; MRS08; JM20].

1.5.4.1 Bag of words model

A simple approach to dealing with variable-length text documents is to interpret them as a **bag of words**, in which we ignore word order. To convert this to a vector from a fixed input space, we first map each word to a **token** from some vocabulary.

To reduce the number of tokens, we often use various pre-processing techniques such as the following: dropping punctuation, converting all words to lower case; dropping common but uninformative words, such as “and” and “the” (this is called **stop word removal**); replacing words with their base form, such as replacing “running” and “runs” with “run” (this is called **word stemming**); etc. For details, see e.g., [BL12], and for some sample code, see [text_preproc_jax.ipynb](#).

Let x_{nt} be the token at location t in the n ’th document. If there are D unique tokens in the vocabulary, then we can represent the n ’th document as a D -dimensional vector $\tilde{\mathbf{x}}_n$, where $\tilde{x}_{nv}$ is the number of times that word v occurs in document n :

$$\tilde{x}_{nv} = \sum_{t=1}^T \mathbb{I}(x_{nt} = v) \quad (1.37)$$

where T is the length of document n . We can now interpret documents as vectors in $\mathbb{R}^D$. This is called the **vector space model** of text [SWY75; TP10].

We traditionally store input data in an $N \times D$ design matrix denoted by $\mathbf{X}$, where D is the number of features. In the context of vector space models, it is more common to represent the input data

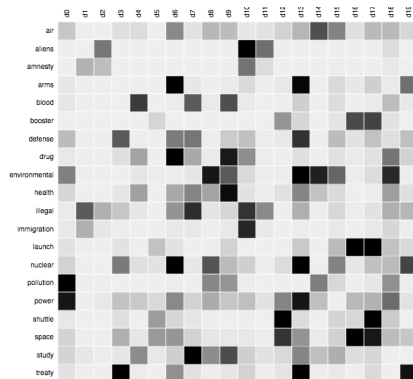


Figure 1.15: Example of a term-document matrix, where raw counts have been replaced by their TF-IDF values (see Section 1.5.4.2). Darker cells are larger values. Used with kind permission of Christoph Carl Kling.

as a $D \times N$ **term frequency matrix**, where TF_{ij} is the frequency of term i in document j . See Figure 1.15 for an illustration.

1.5.4.2 TF-IDF

One problem with representing documents as word count vectors is that frequent words may have undue influence, just because the magnitude of their word count is higher, even if they do not carry much semantic content. A common solution to this is to transform the counts by taking logs, which reduces the impact of words that occur many times within a single document.

To reduce the impact of words that occur many times in general (across all documents), we compute a quantity called the **inverse document frequency**, defined as follows: $\text{IDF}_i \triangleq \log \frac{N}{1 + \text{DF}_i}$, where DF_i is the number of documents with term i . We can combine these transformations to compute the **TF-IDF** matrix as follows:

$$\text{TFIDF}_{ij} = \log(\text{TF}_{ij} + 1) \times \text{IDF}_i \quad (1.38)$$

(We often normalize each row as well.) This provides a more meaningful representation of documents, and can be used as input to many ML algorithms. See [tfidf_demo.ipynb](#) for an example.

1.5.4.3 Word embeddings

Although the TF-IDF transformation improves on raw count vectors by placing more weight on “informative” words and less on “uninformative” words, it does not solve the fundamental problem with the one-hot encoding (from which count vectors are derived), which is that that semantically similar words, such as “man” and “woman”, may be not be any closer (in vector space) than semantically dissimilar words, such as “man” and “banana”. Thus the assumption that points that are close in input space should have similar outputs, which is implicitly made by most prediction models, is invalid.

The standard way to solve this problem is to use **word embeddings**, in which we map each sparse one-hot vector, $\mathbf{x}_{nt} \in \{0, 1\}^V$, to a lower-dimensional dense vector, $\mathbf{e}_{nt} \in \mathbb{R}^K$ using $\mathbf{e}_{nt} = \mathbf{E}\mathbf{x}_{nt}$, where $\mathbf{E} \in \mathbb{R}^{K \times V}$ is learned such that semantically similar words are placed close by. There are many ways to learn such embeddings, as we discuss in Section 20.5.

Once we have an embedding matrix, we can represent a variable-length text document as a **bag of word embeddings**. We can then convert this to a fixed length vector by summing (or averaging) the embeddings:

$$\bar{\mathbf{e}}_n = \sum_{t=1}^T \mathbf{e}_{nt} = \mathbf{E}\tilde{\mathbf{x}}_n \quad (1.39)$$

where $\tilde{\mathbf{x}}_n$ is the bag of words representation from Equation (1.37). We can then use this inside of a logistic regression classifier, which we briefly introduced in Section 1.2.1.5. The overall model has the form

$$p(y = c | \mathbf{x}_n, \boldsymbol{\theta}) = \text{softmax}_c(\mathbf{W}\mathbf{E}\tilde{\mathbf{x}}_n) \quad (1.40)$$

We often use a **pre-trained word embedding** matrix $\mathbf{E}$, in which case the model is linear in $\mathbf{W}$, which simplifies parameter estimation (see Chapter 10). See also Section 15.7 for a discussion of contextual word embeddings.

1.5.4.4 Dealing with novel words

At test time, the model may encounter a completely novel word that it has not seen before. This is known as the **out of vocabulary** or **OOV** problem. Such novel words are bound to occur, because the set of words is an **open class**. For example, the set of proper nouns (names of people and places) is unbounded.

A standard heuristic to solve this problem is to replace all novel words with the special symbol **UNK**, which stands for “unknown”. However, this loses information. For example, if we encounter the word “athazagoraphobia”, we may guess it means “fear of something”, since phobia is a common suffix in English (derived from Greek) to mean “fear of”. (It turns out that athazagoraphobia means “fear of being forgotten about or ignored”.)

We could work at the character level, but this would require the model to learn how to group common letter combinations together into words. It is better to leverage the fact that words have substructure, and then to take as input **subword units** or **wordpieces** [SHB16; Wu+16]; these are often created using a method called **byte-pair encoding** [Gag94], which is a form of data compression that creates new symbols to represent common substrings.

1.5.5 Handling missing data

Sometimes we may have **missing data**, in which parts of the input $\mathbf{x}$ or output y may be unknown. If the output is unknown during training, the example is unlabeled; we consider such semi-supervised learning scenarios in Section 19.3. We therefore focus on the case where some of the input features may be missing, either at training or testing time, or both.

To model this, let $\mathbf{M}$ be an $N \times D$ matrix of binary variables, where $M_{nd} = 1$ if feature d in example n is missing, and $M_{nd} = 0$ otherwise. Let $\mathbf{X}_v$ be the visible parts of the input feature matrix,

corresponding to $M_{nd} = 0$, and $\mathbf{X}_h$ be the missing parts, corresponding to $M_{nd} = 1$. Let $\mathbf{Y}$ be the output label matrix, which we assume is fully observed. If we assume $p(\mathbf{M}|\mathbf{X}_v, \mathbf{X}_h, \mathbf{Y}) = p(\mathbf{M})$, we say the data is **missing completely at random** or **MCAR**, since the missingness does not depend on the hidden or observed features. If we assume $p(\mathbf{M}|\mathbf{X}_v, \mathbf{X}_h, \mathbf{Y}) = p(\mathbf{M}|\mathbf{X}_v, \mathbf{Y})$, we say the data is **missing at random** or **MAR**, since the missingness does not depend on the hidden features, but may depend on the visible features. If neither of these assumptions hold, we say the data is **not missing at random** or **NMAR**.

In the MCAR and MAR cases, we can ignore the missingness mechanism, since it tells us nothing about the hidden features. However, in the NMAR case, we need to model the **missing data mechanism**, since the lack of information may be informative. For example, the fact that someone did not fill out an answer to a sensitive question on a survey (e.g., “Do you have COVID?”) could be informative about the underlying value. See e.g., [LR87; Mar08] for more information on missing data models.

In this book, we will always make the MAR assumption. However, even with this assumption, we cannot directly use a discriminative model, such as a DNN, when we have missing input features, since the input $\mathbf{x}$ will have some unknown values.

A common heuristic is called **mean value imputation**, in which missing values are replaced by their empirical mean. More generally, we can fit a generative model to the input, and use that to **fill in** the missing values. We briefly discuss some suitable generative models for this task in Chapter 20, and in more detail in the sequel to this book, [Mur23].

1.6 Discussion

In this section, we situate ML and this book into a larger context.

1.6.1 The relationship between ML and other fields

There are several subcommunities that work on ML-related topics, each of which have different names. The field of **predictive analytics** is similar to supervised learning (in particular, classification and regression), but focuses more on business applications. **Data mining** covers both supervised and unsupervised machine learning, but focuses more on structured data, usually stored in large commercial databases. **Data science** uses techniques from machine learning and statistics, but also emphasizes other topics, such as data integration, data visualization, and working with domain experts, often in an iterative feedback loop (see e.g., [BS17]). The difference between these areas is often just one of terminology.¹²

ML is also very closely related to the field of **statistics**. Indeed, Jerry Friedman, a famous statistics professor at Stanford, said¹³

[If the statistics field had] incorporated computing methodology from its inception as a fundamental tool, as opposed to simply a convenient way to apply our existing tools, many of the other data related fields [such as ML] would not have needed to exist — they would have been part of statistics. — Jerry Friedman [Fri97b]

12. See <https://developers.google.com/machine-learning/glossary/> for a useful “ML glossary”.

13. Quoted in <https://brenocon.com/blog/2008/12/statistics-vs-machine-learning-fight/>